

```

import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np

# ----- Step 1: Load and Preprocess Data ----- #
print("Loading MNIST dataset...")
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize pixel values (0-255 → 0-1)
x_train = x_train / 255.0
x_test = x_test / 255.0

# Reshape to (28, 28, 1)
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# ----- Step 2: Build CNN Model ----- #
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# ----- Step 3: Compile Model ----- #
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# ----- Step 4: Train Model ----- #
print("🌀 Training model...")
history = model.fit(x_train, y_train, epochs=5, validation_split=0.2)

# ----- Step 5: Evaluate Model ----- #
print("🟩 Evaluating model...")
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)

```

```

print(f"📊 Test accuracy: {test_acc:.4f}")

# ----- Step 6: Visualize Training Results ----- #
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='train_accuracy')
plt.plot(history.history['val_accuracy'], label='val_accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()

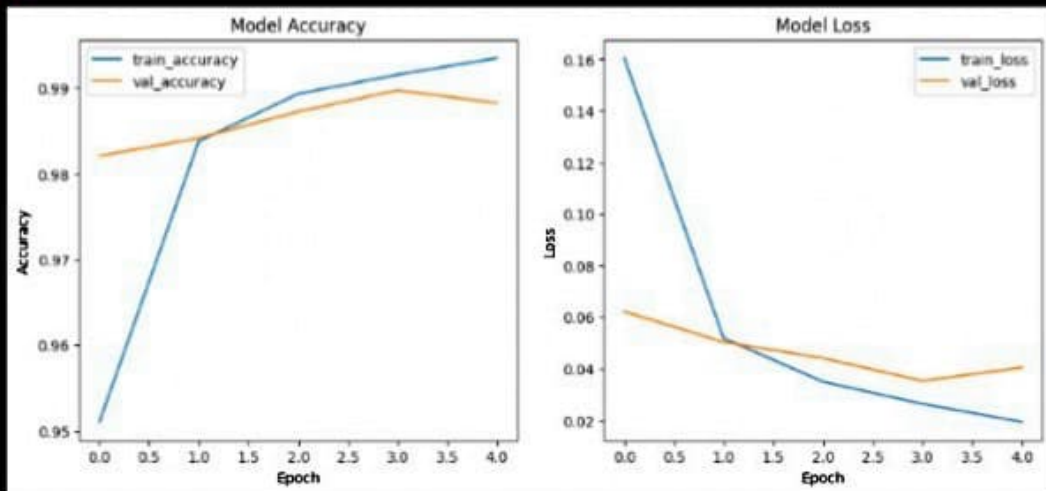
predictions = model.predict(x_test[:10])
predicted_labels = np.argmax(predictions, axis=1)

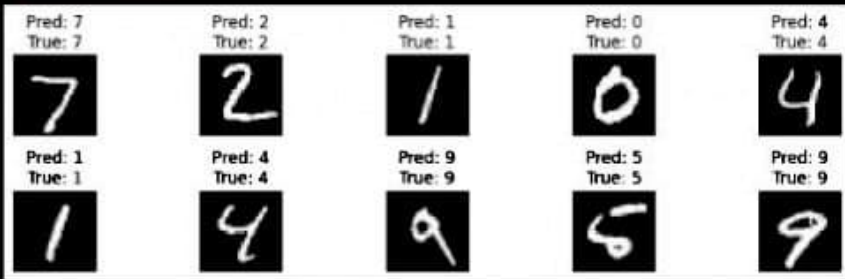
plt.figure(figsize=(10,3))
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.title(f"Pred: {predicted_labels[i]}\nTrue: {y_test[i]}")
    plt.axis('off')
plt.tight_layout()
plt.show()

model.save("mnist_cnn_model.h5")
print(f"📁 Model saved as 'mnist_cnn_model.h5'")

```

```
loading MNIST dataset...
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 — 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape` / `input_dim` argument to a Layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
🔥 Training model...
Epoch 1/5
1500/1500 — 48s 31ms/step - accuracy: 0.8918 - loss: 0.3629 - val_accuracy: 0.9821 - val_loss: 0.0622
Epoch 2/5
1500/1500 — 83s 32ms/step - accuracy: 0.9828 - loss: 0.0543 - val_accuracy: 0.9842 - val_loss: 0.0501
Epoch 3/5
1500/1500 — 47s 31ms/step - accuracy: 0.9891 - loss: 0.0348 - val_accuracy: 0.9872 - val_loss: 0.0441
Epoch 4/5
1500/1500 — 80s 30ms/step - accuracy: 0.9922 - loss: 0.0264 - val_accuracy: 0.9898 - val_loss: 0.0352
Epoch 5/5
1500/1500 — 83s 31ms/step - accuracy: 0.9942 - loss: 0.0169 - val_accuracy: 0.9883 - val_loss: 0.0405
🟢 Evaluating model...
313/313 - 3s - 5ms/step - accuracy: 0.9888 - loss: 0.0340
🟡 Test accuracy: 0.9888
```





WARNING:shsh:You are saving your model as an H5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy. We recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'.
Model saved as 'mist_cnn_model.h5'